

Cloud Storage Services

Cloud Storage (GCP)

Object storage service for storing unstructured data like files and media.

Data stored as objects in buckets.

Ideal for backup, archiving, and big data.

Persistent Disk

Block storage for VMs in Compute Engine or Kubernetes.

Supports SSDs and HDDs.

Suitable for high-performance and durable applications.

Cloud Storage for Firebase

Built for mobile apps.

Secure uploads/downloads with recovery.

Handles network issues with resilience.

Cloud Filestore

Network-attached file storage.

Provides shared file systems for VMs and containers.

Ideal for high IOPS and scalable capacity.

Cloud SQL

Managed relational DB for MySQL & PostgreSQL.

Automated backups and patching.

Ideal for traditional SQL needs.

Cloud Bigtable

NoSQL wide-column database

Supports billions of rows & millions of operations

Ideal for analytics and IoT workloads

Cloud Spanner

Distributed relational DB combining SQL & NoSQL

High availability (99.999% SLA)

Strong consistency, enterprise-grade

Cloud Datastore

Scalable NoSQL document DB

Uses key-value pairs and REST API

Best for scalability without strict consistency

Cloud Memorystore

Managed Redis caching service

Ultra-fast data retrieval

Reduces latency by caching frequent data

6. Cloud Firestore

Serverless NoSQL document DB

Real-time sync for mobile/web apps

Supports offline and live updates

Networking Components

1. Virtual Private Cloud (VPC)

Logically isolated cloud networks

Secure internal communication via Google's network

2. Cloud Load Balancing

Distributes traffic across servers/regions

Supports autoscaling and multiple protocols

(HTTP/S, TCP, UDP)

3. Cloud Armor

Protects apps from DDoS, XSS, SQLi

Offers IP/geolocation rules with global load balancing

4. Cloud CDN

Caches static content at global edge locations

Reduces latency for global delivery

5. Cloud Interconnect

Connects on-premises to Google's network

Options: Dedicated or Partner Interconnect

6. Cloud DNS

High-performance DNS for name-to-IP translation

Supports public and private zones

Billing and Accounting

1. Resource Usage and Billing

Charges based on usage, region, resource type

Billing API helps manage and monitor costs

2. Billing Accounts

Required for most paid services

Projects must be linked to billing accounts

Supports centralized cost management

3. Types of Billing Accounts

Self-Serve – Paid via card/bank; used by individuals and small businesses

Invoiced – Monthly bills; used by enterprises

4. Billing Budgets and Alerts

Set budgets and alerts to monitor spending

Helps avoid overspending and unexpected charges

5. Budget Features

Custom time periods (monthly, quarterly, etc.)

Alerts via email or Pub/Sub when budget limits are exceeded

Aids in cost control and planning

6. Resource Hierarchy Integration

Billing linked to org > folders > projects

Helps track department- or team-level spending

1. Resource and Policy Management

Resource management in cloud computing is a **critical function** that affects:

- **Functionality**
- **Performance**
- **Cost**

Scheduling is the core of resource management and involves deciding how system resources (CPU, memory, storage, I/O, bandwidth) are allocated among users and tasks.

Policies vs Mechanisms:

- **Policies** = The rules or principles guiding decisions.
- **Mechanisms** = The technical methods or tools used to enforce those policies.

Cloud Resource Management (CRM) requires:

- Admission control
- Capacity allocation
- Load balancing
- Energy optimization

- Quality of Service (QoS) guarantees

2. Start-Time Fair Queuing (SFQ)

Start-Time Fair Queuing is a CPU scheduling technique where:

- **Consumers (threads)** of CPU bandwidth are arranged in a **tree structure**.
 - The **root** is the CPU (processor).
 - The **leaves** are threads from applications.

Key Rules:

- If a **VM is inactive**, its bandwidth is distributed among the **active VMs**.
- If an **application within a VM** is inactive, its share goes to other apps in that VM.
- If a **thread** is not runnable, its share is reallocated to other threads of the same application.

This model ensures fair and dynamic allocation of CPU time across various workloads.

3. Two-Level Resource Architecture

The **Two-Level Cloud Controller** has:

- A **cloud-level controller**: manages overall resource allocation and arbitration.
- **Application-level controllers**: manage individual applications, deciding if they need more resources.

Architecture:

- Each application runs on VMs.
- Each application has a controller with:
 - **Monitor** (to observe performance)
 - **Decision** module (to request/release VMs)
 - **Actuator** (to take actions based on decisions)
- The cloud controller receives all requests and allocates/deallocates physical resources accordingly.

Key Insights:

- The system must **avoid instability** caused by rapid reactions to changes.
- Use **upper and lower thresholds** wisely—setting them too close can trigger instability.
- Allocation/deallocation of VMs should be **synchronized with system stability**.